FundaPeças API

Documentação Técnica de Backend

Autor:
DevTeam

Data:
Dezembro de 2025

Conteúdo

1 Introdução **4**
1.1 Arquitetura 4

2 Especificação da Base de Dados **5**
2.1 Tipos Enumerados (Global Enums) 5
2.1.1 Enum: UserRole 5
2.1.2 Enum: PartCondition 5
2.1.3 Enum: ReservationStatus 5
2.1.4 Enum: MovementType 6
2.2 Módulo 1: Utilizadores 7
2.2.1 Modelo: User (Tabela: `users`) 7
2.2.2 Modelo: UserFavorite (Tabela: `user_favorites`) 7
2.3 Módulo 2: Catálogo (Peças) 8
2.3.1 Modelo: Part (Tabela: `parts`) 8
2.3.2 Modelo: Category (Tabela: `categories`) 8
2.3.3 Modelo: Supplier (Tabela: `suppliers`) 8
2.3.4 Modelo: PartReference (Tabela: `part_references`) 8
2.3.5 Modelo: PartImage (Tabela: `part_images`) 9
2.3.6 Modelo: Specification (Tabela: `specifications`) 9
2.3.7 Modelo: PartSpecification (Tabela: `part_specifications`) 9
2.4 Módulo 3: Armazém 10
2.4.1 Modelo: Warehouse (Tabela: `warehouses`) 10
2.4.2 Modelo: Location (Tabela: `locations`) 10
2.5 Módulo 4: Operações 11
2.5.1 Modelo: Reservation (Tabela: `reservations`) 11
2.5.2 Modelo: StockMovement (Tabela: `stock_movements`) 12
2.6 Módulo 5: Auditoria 13
2.6.1 Modelo: AuditLog (Tabela: `audit_logs`) 13
2.6.2 Serviço e Fluxo 13
2.6.3 Ações Registadas 14

3 Documentação de Rotas (Endpoints) **15**
3.1 Módulo: Autenticação (Auth) 15
3.1.1 Arquitetura de Tokens 15
3.1.2 POST /auth/login 15
3.1.3 POST /auth/refresh 16
3.1.4 POST /auth/logout 17
3.1.5 GET /auth/me 17

3.1.6 POST /auth/change-password . . . 18
3.1.7 POST /auth/register . . . 19
3.1.8 POST /auth/admin/reset-password . . . 20
3.2 Módulo: Gestão de Utilizadores (Users) . . . 22
3.2.1 GET /users . . . 22
3.2.2 GET /users/:id . . . 23
3.2.3 PUT /users/:id . . . 24
3.2.4 PUT /users/:id/deactivate . . . 25
3.2.5 PUT /users/:id/activate . . . 26
3.3 Módulo: Favoritos (Favorites) . . . 27
3.3.1 GET /favorites . . . 27
3.3.2 POST /favorites/:partId . . . 27
3.3.3 DELETE /favorites/:partId . . . 28
3.3.4 GET /favorites/:partId/check . . . 28
3.4 Módulo: Catálogo de Peças (Parts) . . . 30
3.4.1 GET /parts . . . 30
3.4.2 GET /parts/:ref/whatsapp-link . . . 31
3.4.3 GET /parts/search . . . 31
3.4.4 GET /parts/:ref/history . . . 32
3.4.5 GET /parts/:ref . . . 33
3.4.6 GET /parts/categories/list . . . 34
3.4.7 GET /parts/specifications/list . . . 34
3.4.8 POST /parts . . . 35
3.4.9 PATCH /parts/:ref . . . 36
3.4.10 DELETE /parts/:ref . . . 37
3.4.11 PATCH /parts/:ref/visibility . . . 38
3.5 Módulo: Reservas (Reservations) . . . 39
3.5.1 Fluxo de Trabalho (Workflow) . . . 39
3.5.2 Motivos de Cancelamento (CancelReason) . . . 39
3.5.3 Integração com Stock Movement . . . 40
3.5.4 POST /reservations . . . 40
3.5.5 GET /reservations . . . 41
3.5.6 GET /reservations/pending . . . 42
3.5.7 GET /reservations/my-assigned . . . 42
3.5.8 GET /reservations/:id . . . 43
3.5.9 POST /reservations/:id/assign . . . 43
3.5.10 PATCH /reservations/:id/status . . . 44
3.5.11 POST /reservations/:id/cancel . . . 45
3.6 Módulo: Movimentos de Stock (Stock Movements) . . . 47
3.6.1 Tipos de Movimento . . . 47
3.6.2 Integração Automática . . . 47
3.6.3 Modelo de Dados . . . 47
3.6.4 POST /stock-movements/entry . . . 47
3.6.5 POST /stock-movements/exit . . . 48
3.6.6 POST /stock-movements/transfer . . . 49
3.6.7 POST /stock-movements/return . . . 50
3.6.8 POST /stock-movements/adjustment . . . 51
3.6.9 GET /stock-movements/part/:partId . . . 52

3.6.10 GET /stock-movements/recent 53
3.6.11 GET /stock-movements/locations 54
3.7 Módulo: Auditoria (Audit Logs) 56
3.7.1 GET /audit-logs 56
3.7.2 GET /audit-logs/:id 57

Capítulo 1

Introdução

Este documento especifica a **FundaPeças API**, um backend RESTful em **Node.js/TypeScript** suportado por **Prisma ORM** e **MariaDB**. A API expõe módulos de autenticação, catálogo de peças, reservas e movimentos de stock, aplicando validação com **Zod**, controlo de acesso por **JWT** e **role-based access control**, e garantindo atomicidade via **transações Prisma**. O esquema de dados segue *soft-delete*, enums para estados de workflow e integrações automáticas entre reservas e movimentos de stock para rastreabilidade. As respostas são em JSON, com convenções de paginação e filtros coerentes entre módulos.

1.1 Arquitetura

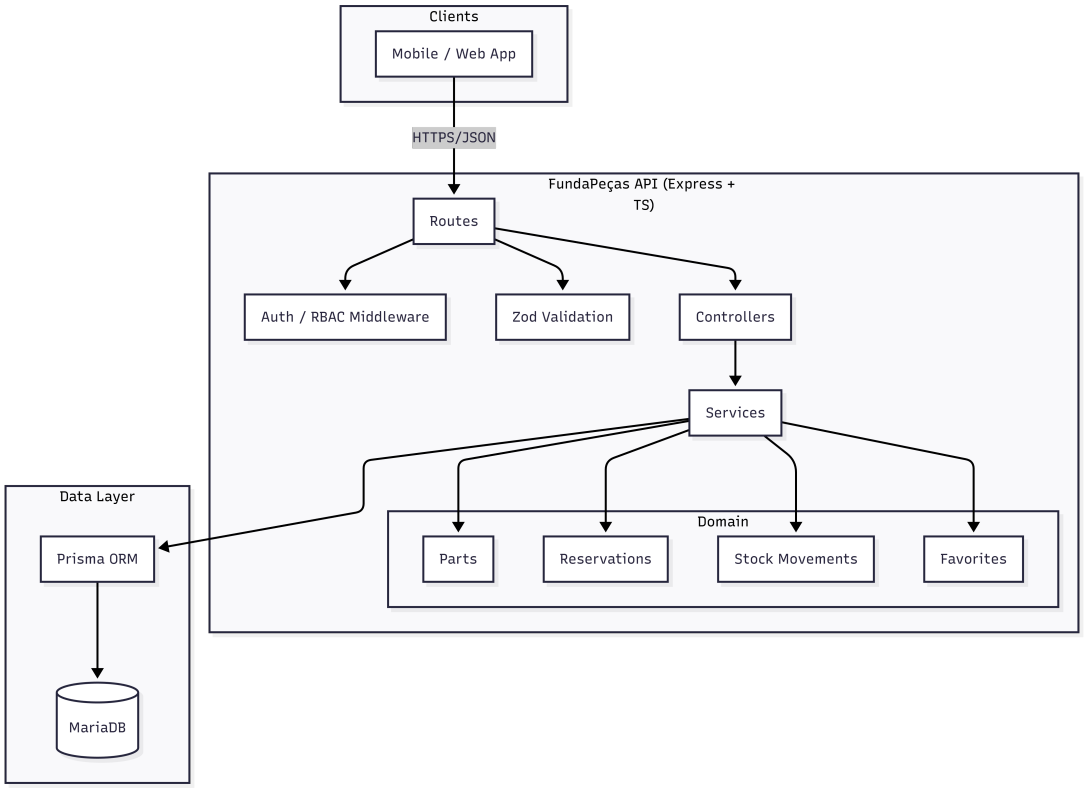


Figura 1.1:

Capítulo 2

Especificação da Base de Dados

2.1 Tipos Enumerados (Global Enums)

Os tipos enumerados definem os estados e categorias fixas permitidos em todo o sistema, garantindo a integridade dos dados ao nível da base de dados (MySQL).

2.1.1 Enum: UserRole

Define os níveis de permissão e perfis de acesso dos utilizadores na plataforma.

Valor	Descrição e Permissões
`ADMIN`	Acesso total ao sistema, gestão de utilizadores e visualização de logs de auditoria.
`SALES`	Gestão comercial: catálogo de peças, fornecedores, clientes e estado de reservas.
`WAREHOUSE`	Gestão operacional: controlo de stock, localizações físicas e movimentos de armazém.
`CLIENT`	Acesso de leitura ao catálogo, gestão de perfil próprio e lista de favoritos.

Tabela 2.2: Definição do enumerado UserRole.

2.1.2 Enum: PartCondition

Especifica o estado físico e a classificação comercial de uma peça no inventário.

Valor	Definição Técnica
`NEW`	Peça nova em folha, sem uso prévio e na embalagem original.
`USED`	Peça de segunda mão, testada e proveniente de desmantelamento ou retoma.
`REFURBISHED`	Peça recondicionada/reparada para cumprir as especificações do fabricante.
`DAMAGED`	Peça com danos visíveis ou falhas, destinada a aproveitamento de componentes.

Tabela 2.4: Definição do enumerado PartCondition.

2.1.3 Enum: ReservationStatus

Controla o fluxo de trabalho (workflow) desde o pedido de reserva até à entrega final.

Valor	Significado no Workflow
PENDING	Pedido submetido pelo cliente, a aguardar validação de stock.
CONFIRMED	Reserva aceite e validada pela equipa de vendas.
IN_PREPARATION	A peça está a ser localizada e embalada pela equipa de armazém.
READY_TO_SHIP	Preparação concluída, aguarda levantamento ou expedição.
COMPLETED	Processo finalizado com a entrega efetiva da peça ao destino.
CANCELLED	Reserva anulada por rutura de stock, desistência ou erro administrativo.

Tabela 2.6: Definição do enumerado ReservationStatus.

2.1.4 Enum: MovementType

Tipifica a natureza das alterações de quantidade ou localização no inventário.

Valor	Natureza do Movimento
ENTRY	Incremento de stock via fornecedor ou entrada de inventário inicial.
EXIT	Decremento de stock por venda direta ou conclusão de reserva.
TRANSFER	Mudança de localização física entre prateleiras ou armazéns.
ADJUSTMENT	Correção de erros detetados durante contagens físicas (inventário).
RETURN	Reentrada de stock devido a devolução de cliente.

Tabela 2.8: Definição do enumerado MovementType.

2.2 Módulo 1: Utilizadores

Este módulo gere a identidade, autenticação e preferências dos utilizadores do sistema FundaPeças.

2.2.1 Modelo: User (Tabela: `users`)

Armazena a informação mestre de todas as contas, definindo os perfis de acesso e credenciais.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
username	username	String	Unique.
email	email	String	Unique.
passwordHash	password_hash	String	Hash da password (Argon2/Bcrypt).
fullName	full_name	String	Nome completo.
role	role	UserRole	Default: `CLIENT`.
isActive	is_active	Boolean	Default: `true`.
createdAt	created_at	DateTime	Default: `now()`.
updatedAt	updated_at	DateTime	Auto-update no Prisma.
passwordUpdatedAt	password_updated_at	DateTime	Default: `now()`.
refreshToken	refresh_token	String?	Opcional (Nullable).

Tabela 2.10: Atributos detalhados do modelo User.

2.2.2 Modelo: UserFavorite (Tabela: `user_favorites`)

Tabela de junção que gere a relação *Many-to-Many* entre utilizadores e peças do catálogo (favoritos).

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
userId	user_id	Int	PK (composta), FK para `User.id`.
partId	part_id	Int	PK (composta), FK para `Part.id`.

Tabela 2.12: Atributos detalhados do modelo UserFavorite.

Nota de Integridade: A tabela `user_favorites` utiliza uma chave primária composta pelos campos `user_id` e `part_id`, garantindo que um utilizador não possa adicionar a mesma peça aos favoritos mais do que uma vez.

2.3 Módulo 2: Catálogo (Peças)

Este módulo constitui o núcleo da plataforma, gerindo o inventário de peças, a sua categorização, fornecedores e especificações técnicas.

2.3.1 Modelo: Part (Tabela: `parts`)

Entidade principal que representa os componentes físicos em stock.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
name	name	String	Nome comercial da peça.
refInternal	ref_internal	String	Unique. Referência interna (SKU).
refOEM	ref_oem	String?	Referência original do fabricante.
description	description	Text	Descrição detalhada (MySQL Text).
price	price	Decimal	(10, 2). Preço unitário.
condition	condition	PartCondition	Default: `USED`.
isVisible	is_visible	Boolean	Default: `true`.
categoryId	category_id	Int	FK para `Category.id`.
supplierId	supplier_id	Int?	FK para `Supplier.id` (Opcional).
locationId	location_id	Int?	FK para `Location.id` (Opcional).
createdAt	created_at	DateTime	Default: `now()`.
updatedAt	updated_at	DateTime	Auto-update no Prisma.
deletedAt	deleted_at	DateTime?	Timestamp para Soft Delete.

Tabela 2.14: Atributos detalhados do modelo Part.

2.3.2 Modelo: Category (Tabela: `categories`)

Gere a hierarquia de categorias (ex: Motor > Alternadores).

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
name	name	String	Nome da categoria.
parentId	parent_id	Int?	Auto-relacionamento (FK para `id`).
deletedAt	deleted_at	DateTime?	Soft Delete.

Tabela 2.16: Atributos detalhados do modelo Category.

2.3.3 Modelo: Supplier (Tabela: `suppliers`)

Entidades externas que fornecem os componentes.

2.3.4 Modelo: PartReference (Tabela: `part_references`)

Gere códigos ou referências extra associados a uma peça única.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
name	name	String	Nome do fornecedor.
nif	nif	String?	Número de Identificação Fiscal.
contact	contact	String?	Informação de contacto.
deletedAt	deleted_at	DateTime?	Soft Delete.

Tabela 2.18: Atributos detalhados do modelo Supplier.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
value	value	String	O código de referência extra.
partId	part_id	Int	FK para `Part.id` (OnDelete: Cascade).

Tabela 2.20: Atributos detalhados do modelo PartReference.

2.3.5 Modelo: PartImage (Tabela: `part_images`)

Armazena os URLs das fotografias das peças.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
url	url	String	URL da imagem (CDN ou local).
isMain	is_main	Boolean	Default: `false`. Define a capa.
partId	part_id	Int	FK para `Part.id`.

Tabela 2.22: Atributos detalhados do modelo PartImage.

2.3.6 Modelo: Specification (Tabela: `specifications`)

Lista global de tipos de especificações (ex: "Voltagem").

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
name	name	String	Nome da especificação.
unit	unit	String?	Unidade de medida (ex: "V", "kg").

Tabela 2.24: Atributos detalhados do modelo Specification.

2.3.7 Modelo: PartSpecification (Tabela: `part_specifications`)

Atribui valores concretos de especificações a cada peça.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
partId	part_id	Int	PK (composta), FK para `Part.id`.
specId	spec_id	Int	PK (composta), FK para `Spec.id`.
value	value	String	Valor real (ex: "12").

Tabela 2.26: Atributos detalhados do modelo PartSpecification.

2.4 Módulo 3: Armazém

Este módulo define a estrutura física onde as peças são armazenadas, permitindo um controlo preciso de inventário e localização.

2.4.1 Modelo: Warehouse (Tabela: `warehouses`)

Representa as unidades físicas de armazenamento (ex: Armazém Principal, Pavilhão B).

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
code	code	String	Unique. Código identificador (ex: "ARM_01").
name	name	String	Nome descritivo do armazém.
deletedAt	deleted_at	DateTime?	Timestamp para Soft Delete.

Tabela 2.28: Atributos detalhados do modelo Warehouse.

2.4.2 Modelo: Location (Tabela: `locations`)

Representa as subdivisões específicas dentro de um armazém, como estantes, prateleiras ou paletes.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
warehouseId	warehouse_id	Int	FK para `Warehouse.id`.
rack	rack	String	Identificação da estante/corredor.
shelf	shelf	String	Identificação da prateleira/nível.
pallet	pallet	String?	Identificação da palete (opcional).
fullCode	full_code	String	Unique. Código completo (ex: "A1-R02-S10").
capacity	capacity	Int	Default: 1. Capacidade de itens.

Tabela 2.30: Atributos detalhados do modelo Location.

Lógica de Negócio: O campo `full_code` é gerado para facilitar a etiquetagem e leitura via código de barras, sendo uma combinação única que impede a duplicação de registos para o mesmo espaço físico.

2.5 Módulo 4: Operações

Este módulo é responsável por gerir o fluxo de negócio, desde a reserva de peças por parte da equipa de vendas até ao registo histórico de movimentações de stock. O sistema implementa um workflow de estados que controla a disponibilidade das peças e o processo de preparação pelo armazém.

2.5.1 Modelo: Reservation (Tabela: `reservations`)

Gere o processo de reserva de componentes, permitindo o acompanhamento do estado do pedido desde a criação pela equipa de vendas até à conclusão da entrega.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
userId	user_id	Int	FK para `User.id`. Quem criou a reserva (SALES).
partId	part_id	Int	FK para `Part.id`. Peça reservada.
assignedToId	assigned_to_id	Int?	FK para `User.id`. Quem assumiu a preparação (WAREHOUSE). Null até ser assumida.
status	status	ReservationStatus	Default: `PENDING`.
notes	notes	String?	Notas adicionais ou observações.
createdAt	created_at	DateTime	Default: `now()`.
updatedAt	updated_at	DateTime	Auto-update no Prisma.

Tabela 2.32: Atributos detalhados do modelo Reservation.

Regra de Negócio - Bloqueio de Peças: Uma peça com reserva ativa (estados `PENDING`, `IN_PREPARATION` ou `READY_TO_SHIP`) não pode ser reservada por outro utilizador. Isto garante que duas pessoas da equipa de vendas não reservem a mesma peça para clientes diferentes.

Fluxo de Trabalho:

1. **SALES** cria reserva → status: `PENDING`, peça fica bloqueada.
2. **WAREHOUSE** assume preparação → status: `IN_PREPARATION`, `assignedToId` preenchido.
3. **WAREHOUSE** conclui preparação → status: `READY_TO_SHIP`.
4. **Entrega ao cliente** → status: `COMPLETED`, cria `StockMovement` EXIT.
5. **Cancelamento** (qualquer estado) → status: `CANCELLED`, peça volta a ficar disponível.

Relações no Modelo User: O modelo `User` possui duas relações com `Reservation`:

- `createdReservations`: Reservas criadas pelo utilizador (role SALES).
- `assignedReservations`: Reservas assumidas pelo utilizador (role WAREHOUSE).

2.5.2 Modelo: StockMovement (Tabela: `stock_movements`)

Registo histórico obrigatório para cada alteração física de stock. Crucial para rastreabilidade.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
partId	part_id	Int	FK para `Part.id`.
userId	user_id	Int	FK para `User.id`. Operador que realizou a ação.
type	type	MovementType	Tipo de fluxo (Entry, Exit, Transfer, etc).
sourceLocId	source_location_id	Int?	FK para `Location.id` (Origem).
destLocId	dest_location_id	Int?	FK para `Location.id` (Destino).
timestamp	timestamp	DateTime	Default: `now()`. Registo temporal exato.

Tabela 2.34: Atributos detalhados do modelo StockMovement.

Regra de Integridade: Para movimentos do tipo `TRANSFER`, ambos os campos de localização (`sourceLocId` e `destLocId`) devem ser preenchidos. Para `ENTRY`, apenas o destino é obrigatório; para `EXIT`, apenas a origem.

2.6 Módulo 5: Auditoria

Este módulo garante a rastreabilidade total das ações críticas realizadas na API, armazenando logs detalhados para fins de segurança, compliance e depuração.

2.6.1 Modelo: AuditLog (Tabela: `audit_logs`)

Regista eventos significativos, alterações em entidades e ações de utilizadores administrativos.

Campo (Prisma)	Coluna (DB)	Tipo	Restrições / Notas
id	id	Int	PK, Autoincrement.
userId	user_id	Int	FK para `User.id`. Autor da ação.
action	action	String	Código curto da ação (ex: "PART_CREATE"). É normalizado para maiúsculas.
entity	entity	String	Nome da entidade afetada (ex: "PART", "RESERVATION").
entityId	entityId	String	ID do registo afetado (armazenado como String para aceitar UUIDs ou ints).
details	details	Json	Dados contextuais (opcional).
timestamp	timestamp	DateTime	Default: `now()`. Data/Hora exata.

Tabela 2.36: Atributos detalhados do modelo AuditLog.

Uso de Dados JSON: O campo `details` utiliza o tipo nativo JSON do MySQL para permitir o armazenamento flexível de metadados, como o estado do objeto antes e depois da modificação, sem necessidade de alterar o esquema.

2.6.2 Serviço e Fluxo

- O serviço central `auditLogService` exposto em `src/services/auditLog.service.ts` oferece dois métodos: `record()` (escrita dentro de transações Prisma) e `getLogs()` / `getLogById()` (consulta paginada com filtros).
- Os controladores/rotas (`/audit-logs`) são protegidos por `authenticate` + `requireRole(ADMIN)`, garantindo que apenas administradores consultam os registos.
- Filtros suportados: `page`, `pageSize` (máx. 200), `userId`, `action`, `entity`, `entityId`, `search` (full-text simplificado) e intervalo temporal `dateFrom/dateTo` (ISO 8601).
- O serviço inclui automaticamente os dados mínimos do utilizador (`id`, `username`, `fullName`, `role`) para cada log, facilitando investigações rápidas.
- Controlos de erro: validação via **Zod** (400 para filtros inválidos), 404 quando um `id` não existe e 500 para falhas inesperadas.

2.6.3 Ações Registadas

O catálogo atual cobre as operações críticas de peças, reservas, stock e gestão de utilizadores. Cada ação mapeia diretamente para um endpoint ou serviço específico.

- **PART_CREATE / PART_UPDATE / PART_DELETE / PART_VISIBILITY**: respetivamente POST, PATCH, DELETE e PATCH `/parts/:ref/visibility`. Capturam os campos alterados e o operador responsável.
- **RESERVATION_CREATE / RESERVATION_ASSIGN / RESERVATION_STATUS / RESERVATION_CANCEL**: cobrem o ciclo de vida completo da reserva, incluindo motivos e logística de devolução.
- **STOCK_ENTRY / STOCK_EXIT / STOCK_TRANSFER / STOCK_RETURN / STOCK_ADJUSTMENT**: disparados por cada fluxo do `stockMovementService`, garantindo rastreabilidade física do inventário.
- **USER_CREATE / USER_UPDATE / USER_DEACTIVATE / USER_ACTIVATE** ações administrativas sobre contas.
- **USER_PASSWORD_CHANGE / USER_PASSWORD_RESET**: distingue operações auto-serviço versus reset feito por administrador.

Capítulo 3

Documentação de Rotas (Endpoints)

A FundaPeças API segue os princípios REST, utilizando o formato JSON para comunicação e códigos de estado HTTP padronizados.

3.1 Módulo: Autenticação (Auth)

Este módulo gere toda a autenticação e autorização do sistema. Utiliza **JWT** (JSON Web Tokens) com estratégia de *Access Token* + *Refresh Token* com rotação automática. O sistema implementa validação robusta de inputs através de validadores personalizados.

3.1.1 Arquitetura de Tokens

- **Access Token**: Validade de 30 minutos. Contém `id`, `username` e `role`.
- **Refresh Token**: Validade de 7 dias. Armazenado na base de dados para permitir revogação.
- **Rotação**: A cada refresh, um novo par de tokens é gerado e o anterior é invalidado.

3.1.2 POST /auth/login

Autenticação de Utilizador

Autentica um utilizador e retorna os tokens de acesso.

Segurança: Endpoint público com *Rate Limiting* (5 tentativas por 15 minutos por IP).

Regras de Validação:

- `username`: String obrigatória, 3-30 caracteres, apenas letras, números e underscores.
- `password`: String obrigatória, máximo 100 caracteres.

Exemplo de Request:

```
{
  "username": "admin",
  "password": "Password123"
}
```

Resposta de Sucesso (200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "username": "admin",
    "fullName": "Administrador",
    "role": "ADMIN"
  }
}
```

Gestão de Erros:

- **400 Bad Request**: Falha na validação (campos em falta ou formato inválido).
- **401 Unauthorized**: Credenciais inválidas (utilizador não encontrado ou password incorreta).
- **429 Too Many Requests**: Limite de tentativas excedido.
- **500 Internal Server Error**: Erro inesperado.

3.1.3 POST /auth/refresh

Renovar Tokens

Gera um novo par de tokens usando o refresh token válido.

Segurança: Endpoint público com *Rate Limiting* (10 tentativas por hora por IP).

Regras de Validação:

- `refreshToken`: String obrigatória, máximo 1000 caracteres.

Exemplo de Request:

```
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

Resposta de Sucesso (200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

Gestão de Erros:

- **400 Bad Request**: Refresh token não fornecido ou inválido.
- **401 Unauthorized**: Token expirado ou já não é válido (revogado).
- **403 Forbidden**: Utilizador inativo.
- **500 Internal Server Error**: Erro inesperado.

3.1.4 POST /auth/logout

Terminar Sessão

Invalida o refresh token do utilizador autenticado.

Segurança: Requer *Bearer Token* válido.

Resposta de Sucesso (200 OK):

```
{
  "message": "Logged out"
}
```

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **500 Internal Server Error**: Erro inesperado.

3.1.5 GET /auth/me

Obter Perfil do Utilizador Autenticado

Retorna os dados do utilizador autenticado.

Segurança: Requer *Bearer Token* válido.

Resposta de Sucesso (200 OK):

```
{
  "id": 1,
  "username": "admin",
  "email": "admin@example.com",
  "fullName": "Administrador",
  "role": "ADMIN",
  "isActive": true,
  "createdAt": "2025-12-03T03:41:57.000Z",
  "updatedAt": "2025-12-24T10:00:00.000Z"
}
```

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **500 Internal Server Error**: Erro inesperado.

3.1.6 POST /auth/change-password

Alterar Própria Password (Admin)

Permite ao administrador alterar a sua própria password, fornecendo a password atual como verificação.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Nota Importante: Este endpoint é exclusivo para administradores. Utilizadores com outros roles (SALES, WAREHOUSE, CLIENT) não têm permissão para alterar a sua própria password — apenas um administrador pode fazê-lo através do endpoint: `/auth/admin/reset-password`.

Regras de Validação:

- `oldPassword`: String obrigatória, máximo 100 caracteres (password atual do admin).
- `newPassword`: String obrigatória, 6-100 caracteres, pelo menos uma letra minúscula, uma maiúscula e um número.

Exemplo de Request:

```
{
  "oldPassword": "OldPassword123",
  "newPassword": "NewPassword456"
}
```

Resposta de Sucesso (200 OK):

```
{
  "message": "Password changed"
}
```

Efeitos Colaterais:

- O campo `passwordUpdatedAt` é atualizado, invalidando access tokens emitidos antes da alteração.
- O refresh token é removido, forçando novo login.

Gestão de Erros:

- **400 Bad Request**: Falha na validação da nova password.
- **401 Unauthorized**: Password antiga incorreta ou token inválido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador não encontrado.
- **500 Internal Server Error**: Erro inesperado.

3.1.7 POST /auth/register

Registar Novo Utilizador

Cria um novo utilizador no sistema.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Regras de Validação:

- `username`: String obrigatória, 3-30 caracteres, apenas letras, números e underscores.
- `email`: Opcional, formato de email válido, máximo 255 caracteres.
- `fullName`: String obrigatória, 2-100 caracteres.
- `password`: String obrigatória, 6-100 caracteres, pelo menos uma letra minúscula, uma maiúscula e um número.
- `role`: Obrigatório, deve ser um de: `ADMIN`, `SALES`, `WAREHOUSE`, `CLIENT`.

Exemplo de Request:

```
{
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Silva",
  "password": "UserPass123",
  "role": "CLIENT"
}
```

Resposta de Sucesso (201 Created):

```
{
  "id": 5,
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Silva",
  "role": "CLIENT"
}
```

Gestão de Erros:

- **400 Bad Request**: Falha na validação dos campos ou role inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **409 Conflict**: Username ou email já existem no sistema.
- **500 Internal Server Error**: Erro inesperado.

3.1.8 POST /auth/admin/reset-password

Reset de Password por Administrador

Permite ao administrador redefinir a password de qualquer utilizador sem necessidade da password antiga.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Regras de Validação:

- `userId`: Inteiro positivo obrigatório.
- `newPassword`: String obrigatória, 6-100 caracteres, pelo menos uma letra minúscula, uma maiúscula e um número.

Exemplo de Request:

```
{
  "userId": 5,
  "newPassword": "NewSecurePass789"
}
```

Resposta de Sucesso (200 OK):

```
{
  "message": "Password reset successfully",
  "userId": 5
}
```

Restrições:

- O administrador não pode usar este endpoint para redefinir a sua própria password (deve usar `/change-password`).

Efeitos Colaterais:

- O campo `passwordUpdatedAt` é atualizado, invalidando access tokens do utilizador alvo.
- O refresh token do utilizador alvo é removido, forçando novo login.

Gestão de Erros:

- **400 Bad Request**: Falha na validação ou tentativa de redefinir própria password.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador alvo não encontrado.
- **500 Internal Server Error**: Erro inesperado.

3.2 Módulo: Gestão de Utilizadores (Users)

Este módulo gere todas as operações CRUD sobre utilizadores do sistema. Todos os endpoints requerem autenticação e role `ADMIN`.

3.2.1 GET /users

Listar Todos os Utilizadores

Retorna a lista de todos os utilizadores do sistema com suporte a filtros.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros de Query (Opcionais):

- `role`: Filtrar por role (`ADMIN`, `SALES`, `WAREHOUSE`, `CLIENT`).
- `isActive`: Filtrar por estado ativo (`true` ou `false`).

Exemplo de Request:

```
GET /users?role=ADMIN&isActive=true
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Resposta de Sucesso (200 OK):


```
[
  {
    "id": 1,
    "username": "admin",
    "email": "admin@example.com",
    "fullName": "Administrador",
    "role": "ADMIN",
    "isActive": true,
    "createdAt": "2025-12-03T03:41:57.000Z",
    "updatedAt": "2025-12-24T10:00:00.000Z"
  },
  {
    "id": 2,
    "username": "jose_silva",
    "email": "jose@example.com",
    "fullName": "José Silva",
    "role": "CLIENT",
    "isActive": true,
    "createdAt": "2025-12-10T14:30:00.000Z",
    "updatedAt": "2025-12-10T14:30:00.000Z"
  }
]
```

Nota: A password nunca é retornada nos dados do utilizador.

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **500 Internal Server Error**: Erro inesperado.

3.2.2 GET /users/:id

Obter Utilizador por ID

Retorna os dados de um utilizador específico.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros de URL:

- `id`: ID numérico do utilizador (obrigatório).

Exemplo de Request:

```
GET /users/5
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Resposta de Sucesso (200 OK):

```
{
  "id": 5,
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Silva",
  "role": "CLIENT",
  "isActive": true,
  "createdAt": "2025-12-10T14:30:00.000Z",
  "updatedAt": "2025-12-10T14:30:00.000Z"
}
```

Gestão de Erros:

- **400 Bad Request**: ID inválido (não é um número).
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador não encontrado.
- **500 Internal Server Error**: Erro inesperado.

3.2.3 PUT /users/:id

Atualizar Utilizador

Atualiza os dados de um utilizador existente.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros de URL:

- `id`: ID numérico do utilizador (obrigatório).

Campos Atualizáveis (pelo menos um obrigatório):

- `username`: String, 3-30 caracteres, apenas letras, números e underscores.
- `email`: Formato de email válido, máximo 255 caracteres.
- `fullName`: String, 2-100 caracteres.
- `role`: Um de: `ADMIN`, `SALES`, `WAREHOUSE`, `CLIENT`.

Exemplo de Request:

```
PUT /users/5
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "fullName": "José Manuel Silva",
  "role": "SALES"
}
```

Resposta de Sucesso (200 OK):

```
{
  "id": 5,
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Manuel Silva",
  "role": "SALES",
  "isActive": true,
  "createdAt": "2025-12-10T14:30:00.000Z",
  "updatedAt": "2026-01-21T10:15:00.000Z"
}
```

Gestão de Erros:

- **400 Bad Request**: Nenhum campo fornecido ou validação falhou.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador não encontrado.
- **409 Conflict**: Username ou email já existem.
- **500 Internal Server Error**: Erro inesperado.

3.2.4 PUT /users/:id/deactivate

Desativar Utilizador

Desativa um utilizador, impedindo-o de fazer login.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros de URL:

- `id`: ID numérico do utilizador (obrigatório).

Exemplo de Request:

```
PUT /users/5/deactivate
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Resposta de Sucesso (200 OK):

```
{
  "id": 5,
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Silva",
  "role": "CLIENT",
  "isActive": false,
  "createdAt": "2025-12-10T14:30:00.000Z",
  "updatedAt": "2026-01-21T10:20:00.000Z"
}
```

Efeitos Colaterais:

- O utilizador não conseguirá fazer login enquanto estiver inativo.
- Tentativas de refresh token falharão com erro 403.

Gestão de Erros:

- **400 Bad Request**: ID inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador não encontrado.
- **500 Internal Server Error**: Erro inesperado.

3.2.5 PUT /users/:id/activate

Ativar Utilizador

Reativa um utilizador previamente desativado.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros de URL:

- `id`: ID numérico do utilizador (obrigatório).

Exemplo de Request:

```
PUT /users/5/activate
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Resposta de Sucesso (200 OK):

```
{
  "id": 5,
  "username": "jose_silva",
  "email": "jose@example.com",
  "fullName": "José Silva",
  "role": "CLIENT",
  "isActive": true,
  "createdAt": "2025-12-10T14:30:00.000Z",
  "updatedAt": "2026-01-21T10:25:00.000Z"
}
```

Gestão de Erros:

- **400 Bad Request**: ID inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão (não é ADMIN).
- **404 Not Found**: Utilizador não encontrado.
- **500 Internal Server Error**: Erro inesperado.

3.3 Módulo: Favoritos (Favorites)

Este módulo permite aos utilizadores gerir a sua lista de peças favoritas. Utiliza a tabela de junção `user_favorites` para implementar a relação Many-to-Many entre utilizadores e peças.

3.3.1 GET /favorites

Listar Favoritos do Utilizador

Retorna a lista completa de peças favoritas do utilizador autenticado.

Segurança: Requer *Bearer Token*. Qualquer *UserRole* pode aceder.

Objetos Incluídos na Resposta: Cada peça favorita inclui: `category`, `location` e `images`.

Resposta de Sucesso (200 OK):

```
[
  {
    "id": 1,
    "name": "Alternador Bosch",
    "refInternal": "ALT-001",
    "price": "89.99",
    "condition": "USED",
    "category": { "id": 1, "name": "Alternadores" },
    "location": { "id": 1, "fullCode": "A-01-01" },
    "images": [{ "id": 1, "url": "...", "isMain": true }]
  }
]
```

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **500 Internal Server Error**: Erro inesperado.

3.3.2 POST /favorites/:partId

Adicionar Peça aos Favoritos

Adiciona uma peça à lista de favoritos do utilizador autenticado.

Segurança: Requer *Bearer Token*. Qualquer *UserRole* pode aceder.

Parâmetros de URL:

- `partId`: ID da peça (inteiro positivo).

Resposta de Sucesso (201 Created):

```
{
  "message": "Added to favorites",
  "partId": 1
}
```

Gestão de Erros:

- **400 Bad Request**: ID da peça inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **404 Not Found**: Peça não encontrada.
- **409 Conflict**: Peça já está nos favoritos.
- **500 Internal Server Error**: Erro inesperado.

3.3.3 DELETE /favorites/:partId

Remover Peça dos Favoritos

Remove uma peça da lista de favoritos do utilizador autenticado.

Segurança: Requer *Bearer Token*. Qualquer *UserRole* pode aceder.

Parâmetros de URL:

- `partId`: ID da peça (inteiro positivo).

Resposta de Sucesso (200 OK):

```
{
  "message": "Removed from favorites"
}
```

Gestão de Erros:

- **400 Bad Request**: ID da peça inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **404 Not Found**: Favorito não encontrado (peça não estava nos favoritos).
- **500 Internal Server Error**: Erro inesperado.

3.3.4 GET /favorites/:partId/check

Verificar se Peça é Favorita

Verifica se uma peça específica está na lista de favoritos do utilizador.

Segurança: Requer *Bearer Token*. Qualquer *UserRole* pode aceder.

Parâmetros de URL:

- `partId`: ID da peça (inteiro positivo).

Resposta de Sucesso (200 OK):

```
{
  "partId": 1,
  "isFavorite": true
}
```

Gestão de Erros:

- **400 Bad Request**: ID da peça inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **500 Internal Server Error**: Erro inesperado.

3.4 Módulo: Catálogo de Peças (Parts)

Este módulo gere o inventário. O sistema utiliza **Zod** para validação rigorosa de entradas e controla a densidade de armazenamento via lógica de capacidade de localização.

3.4.1 GET /parts

Listar Peças

Retorna a listagem completa de todas as peças registadas no sistema que não foram alvo de eliminação lógica (*soft-delete*).

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Regras de Negócio:

- Apenas são retornados registos onde `deletedAt` é nulo.
- A resposta inclui automaticamente os objetos relacionados: `category`, `location`, `images`, `specifications` (com os detalhes de cada `spec`) e `subReferences`.

Exemplo de Request:

```
GET /parts
Authorization: Bearer <token>
```

Resposta de Sucesso (200 OK):


```
[
  {
    "id": 1,
    "name": "Alternador 12V",
    "refInternal": "ALT-001",
    "category": { "id": 2, "name": "Elétrico" },
    "location": { "id": 10, "fullCode": "ARM1-A-01-P1" },
    "images": [ { "id": 1, "url": "https://cdn.exemplo.com/p1.jpg" } ],
    "specifications": [ { "id": 1, "value": "12V", "spec": { "name": "Voltagem" } } ]
    "subReferences": [ { "id": 1, "reference": "REF-ABC-123" } ]
  }
]
```


Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Utilizador não tem permissão de acesso.
- **500 Internal Server Error**: Erro inesperado ao processar a listagem.

3.4.2 GET /parts/:ref/whatsapp-link

Gerar Link de Reserva via WhatsApp

Produz um link curto (`wa.me`) que pré-preenche uma mensagem de reserva para a peça indicada, facilitando o contacto rápido entre vendas e o cliente.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`. O endpoint apenas lê dados públicos da peça.

Pré-requisitos de Configuração: A variável `WHATSAPP_SALES_NUMBER` deve estar definida; o serviço sanitiza o valor para conter apenas dígitos antes de gerar o link (ver Seção **??**).

Resposta de Sucesso (200 OK):

```
{
  "url": "https://wa.me/351912345678?text=Ola!%20Gostaria%20de..."
}
```

Mensagem Predefinida: `"Olá! Gostaria de reservar a peça <nome> (referência interna: <ref>). Podem ajudar?"` — codificada em URL para ser compatível com clientes móveis e desktop.

Gestão de Erros:

- **404 Not Found**: Peça inexistente ou removida (`PartsService.getPartSummary`).
- **500 Internal Server Error**: Configuração de WhatsApp inválida ou falha inesperada na geração do link.

3.4.3 GET /parts/search

Pesquisar Peças

Retorna uma lista paginada de peças com filtros opcionais (texto, categoria, condição, preço, localização, visibilidade) e ordenação configurável.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros de Query (Zod):

- `text` (opcional): pesquisa em `name`, `refInternal`, `refOEM` e `subReferences.value`.
- `categoryId`, `condition`, `locationId` (opcionais).
- `priceMin`, `priceMax` (opcionais; validação `priceMin` $\leq$ `priceMax`).
- `isVisible` (booleano opcional).

- Paginação: `page` (default: 1), `pageSize` (default: 20, máx: 100).
- Ordenação: `sortBy` {name, price, createdAt, updatedAt, refInternal} (default: createdAt); `sortOrder` {asc, desc} (default: desc).

Exemplo de Request:

```
GET /parts/search?text=alt&categoryId=2&isVisible=true
Authorization: Bearer <token>
```

Resposta de Sucesso (200 OK):

```
{
  "items": [ { ... part with category, location, specifications+spec, ... } ],
  "total": 2,
  "page": 1,
  "pageSize": 20,
  "totalPages": 1
}
```

Gestão de Erros:

- **400 Bad Request**: Falha na validação (Zod) dos parâmetros.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao processar a pesquisa.

3.4.4 GET /parts/:ref/history

Histórico de Peça

Retorna os detalhes da peça (por `refInternal`) e o histórico associado de movimentos de stock e reservas, para fins de auditoria.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros:

- `:ref`: Referência interna da peça.

Conteúdo da Resposta (200 OK):

```
{
  "part": { ... category, location, specifications+spec, subReferences ... },
  "movements": [
    {
      "id": 1,
      "type": "ENTRY",
      "timestamp": "...",
```

```
      "user": { "id": 3, "username": "u1", "fullName": "Op A", "role": ... },
      "sourceLoc": null,
      "destLoc": { "id": 5, "fullCode": "ARM1-A-01-P1", ... }
    },
    ...
  ],
  "reservations": [
    {
      "id": 10,
      "status": "PENDING",
      "user": { "id": 2, "username": "sales1", "fullName": "Sales One", ... },
      "assignedTo": { "id": 4, "username": "wh1", "fullName": ..." },
      "returnLocation": { "id": 5, "fullCode": "ARM1-A-01-P1", ... }
    },
    ...
  ]
}
```

Gestão de Erros:

- **404 Not Found**: Peça não encontrada.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou sem permissões.
- **500 Internal Server Error**: Erro inesperado ao obter histórico.

3.4.5 GET /parts/:ref

Consultar Peça por Referência

Obtém os detalhes completos de uma peça através da sua `refInternal`.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros:

- `:ref`: Referência interna da peça.

Resposta de Sucesso (200 OK):

```
{
  "id": 1,
  "name": "Alternador 12V",
  "refInternal": "ALT-001",
  "refOEM": "1234",
  "price": "85.00",
  "condition": "USED",
  "category": { "id": 2, "name": "Elétrico" },
  "location": { "id": 10, "fullCode": "ARM1-A-01-P1" },
```

```
  "images": [ { "id": 1, "url": "..." } ],
  "specifications": [ { "spec": { "name": "Voltagem" }, "value": "12V" } ],
  "subReferences": [ { "id": 1, "value": "REF-ABC-123" } ]
}
```

Gestão de Erros:

- **404 Not Found**: Peça não encontrada.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao obter a peça.

3.4.6 GET /parts/categories/list

Listar Categorias

Retorna todas as categorias ativas (não eliminadas) com a relação hierárquica por `parentId`.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Resposta de Sucesso (200 OK):

```
[
  { "id": 1, "name": "Motor", "parentId": null },
  { "id": 2, "name": "Componentes Elétricos", "parentId": 1 }
]
```

Gestão de Erros:

- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao listar categorias.

3.4.7 GET /parts/specifications/list

Listar Especificações

Retorna a lista de especificações disponíveis (catálogo de specs) ordenadas por nome.

Segurança: Requer *Bearer Token* e role `CLIENT`, `SALES`, `WAREHOUSE` ou `ADMIN`.

Resposta de Sucesso (200 OK):

```
[
  { "id": 1, "name": "Voltagem", "unit": "V" },
  { "id": 2, "name": "Amperagem", "unit": "A" }
]
```

Gestão de Erros:

- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao listar especificações.

3.4.8 POST /parts

Criar Nova Peça

Regista uma nova peça com validação de esquema e verifica a capacidade da localização. Regista automaticamente um movimento de stock do tipo ENTRY.

Segurança: Requer *Bearer Token* e role `WAREHOUSE`, `SALES` ou `ADMIN`.

Corpo (Zod):

- `name` (string), `refInternal` (string), `refOEM?` (string|null), `description?` (string|null)
- `price` (número ≥ 0), `condition` $\in$ {NEW, USED, REFURBISHED, DAMAGED}
- `categoryId` (int>0), `locationId` (int>0)
- `specifications?`: lista de {`specId` (int>0), `value` (string)}
- `subReferences?`: lista de strings

Regras de Negócio:

- Falha se a localização não existir ou não tiver capacidade disponível.
- `refInternal` deve ser única.
- Movimento ENTRY é registado após criar a peça.

Exemplo de Request:

```
POST /parts
Authorization: Bearer <token>
Content-Type: application/json
{
  "name": "Alternador 12V",
  "refInternal": "ALT-001",
  "refOEM": "1234",
  "price": 85,
  "condition": "USED",
  "categoryId": 2,
  "locationId": 5,
  "specifications": [{ "specId": 1, "value": "12V" }],
  "subReferences": ["REF-ABC-123"]
}
```

Resposta de Sucesso (201 Created):

```
{
  "id": 1,
  "name": "Alternador 12V",
  "refInternal": "ALT-001",
  "condition": "USED",
  "category": { ... },
  "location": { ... },
  "specifications": [ { "spec": { "name": "Voltagem" }, "value": "12V" } ],
  "subReferences": [ { "value": "REF-ABC-123" } ],
  "images": []
}
```

Gestão de Erros:

- **400 Bad Request**: Validação Zod ou referência inválida (FK).
- **404 Not Found**: Localização não encontrada.
- **409 Conflict**: `refInternal` duplicada ou localização sem capacidade.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou sem permissões.
- **500 Internal Server Error**: Erro inesperado ao criar a peça.

3.4.9 PATCH /parts/:ref

Atualizar Peça

Atualiza parcialmente uma peça identificada pela sua `refInternal`, exigindo pelo menos um campo no corpo do pedido.

Segurança: Requer *Bearer Token* e role `WAREHOUSE`, `SALES` ou `ADMIN`.

Corpo (Zod - parcial):

- Qualquer subconjunto de: `name`, `refInternal`, `refOEM?`, `description?`, `price` (≥ 0), `condition` $\in$ {NEW, USED, REFURBISHED, DAMAGED}, `categoryId`, `locationId`.
- `specifications?`: lista de {`specId` (int>0), `value` (string)} — substitui todas as anteriores.
- `subReferences?`: lista de strings — substitui todas as anteriores.

Regras de Negócio:

- Falha se a nova localização não existir ou não tiver capacidade disponível.
- `refInternal` deve permanecer única (conflito gera 409).
- Quando `specifications` ou `subReferences` são enviados, os conjuntos existentes são removidos e recriados.

Exemplo de Request:

```
PATCH /parts/ALT-001
Authorization: Bearer <token>
Content-Type: application/json
{
  "price": 90,
  "locationId": 5,
  "specifications": [{ "specId": 1, "value": "12V" }],
  "subReferences": ["REF-NEW-123"]
}
```

Resposta de Sucesso (200 OK):

```
{
  "id": 1,
  "name": "...",
  "refInternal": "ALT-001",
  "price": "90.00",
  "category": { ... },
  "location": { ... },
  "specifications": [ { "spec": { "name": "Voltagem" }, "value": "12V" } ],
  "subReferences": [ { "value": "REF-NEW-123" } ],
  "images": [ ... ]
}
```

Gestão de Erros:

- **400 Bad Request**: Validação Zod ou FK inválida.
- **404 Not Found**: Peça ou localização não encontrada.
- **409 Conflict**: `refInternal` duplicada ou localização sem capacidade.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou sem permissões.
- **500 Internal Server Error**: Erro inesperado ao atualizar a peça.

3.4.10 DELETE /parts/:ref

Eliminar Peça (Soft-Delete)

Efetua *soft-delete* da peça identificada por `refInternal`, marcando `deletedAt` e definindo `isVisible=false`.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Parâmetros:

- `:ref`: Referência interna da peça.

Resposta de Sucesso (204 No Content): Sem corpo.

Gestão de Erros:

- **404 Not Found**: Peça não encontrada (ou já eliminada).
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao eliminar a peça.

3.4.11 PATCH /parts/:ref/visibility

Atualizar Visibilidade da Peça

Atualiza exclusivamente o campo `isVisible` da peça identificada pela sua `refInternal`.

Segurança: Requer *Bearer Token* e role `WAREHOUSE`, `SALES` ou `ADMIN`.

Parâmetros:

- `:ref`: Referência interna da peça.

Corpo (Zod):

```
{ "isVisible": true | false }
```

Resposta de Sucesso (200 OK):

```
{
  "id": 1,
  "refInternal": "ALT-001",
  "isVisible": false,
  "category": { ... },
  "location": { ... },
  "images": [ ... ],
  "specifications": [ { "spec": { "name": "Voltagem" }, "value": "12V" } ],
  "subReferences": [ ... ]
}
```

Gestão de Erros:

- **400 Bad Request**: Falha na validação do corpo.
- **404 Not Found**: Peça não encontrada.
- **401 Unauthorized** / **403 Forbidden**: Token inválido ou falta de permissões.
- **500 Internal Server Error**: Erro inesperado ao atualizar visibilidade.

3.5 Módulo: Reservas (Reservations)

Este módulo gere o fluxo de trabalho de reservas de peças, desde o pedido inicial pela equipa de vendas até à confirmação final pelo cliente. Implementa um sistema de workflow com estados bem definidos, controlo de permissões por role e integração automática com o módulo de movimentos de stock.

3.5.1 Fluxo de Trabalho (Workflow)

Estados da Reserva

1. **PENDING**: Reserva criada por SALES, aguarda processamento.
2. **IN_PREPARATION**: WAREHOUSE assumiu a preparação.
3. **READY_TO_SHIP**: Peça preparada, pronta para entrega.
4. **COMPLETED**: Peça entregue ao cliente (EXIT automático registado).
5. **CONFIRMED**: Cliente confirmou receção (estado final positivo).
6. **CANCELLED**: Reserva cancelada (com motivo obrigatório).

Transições Permitidas:

- PENDING → IN_PREPARATION, CANCELLED
- IN_PREPARATION → READY_TO_SHIP, CANCELLED
- READY_TO_SHIP → COMPLETED, CANCELLED
- COMPLETED → CONFIRMED, CANCELLED (devolução)
- CONFIRMED → (estado final)
- CANCELLED → (estado final)

3.5.2 Motivos de Cancelamento (CancelReason)

Tipos de Cancelamento

- **DESIST**: Cliente desistiu antes da entrega. Peça permanece no armazém.
- **RETURN**: Devolução normal após entrega. Peça volta para localização escolhida.
- **DAMAGED_RETURN**: Devolução com peça danificada. Peça fica invisível (condition=DAMAGED, isVisible=false).

Regras de Cancelamento:

- Antes de COMPLETED: Apenas `DESIST` é permitido.
- Após COMPLETED: Apenas `RETURN` ou `DAMAGED_RETURN` são permitidos.
- `RETURN` requer `returnLocationId` obrigatório.
- Reservas CONFIRMED ou já CANCELLED não podem ser canceladas.

3.5.3 Integração com Stock Movement

Movimentos Automáticos

- **COMPLETED**: Regista automaticamente movimento `EXIT` (peça sai do armazém).
- **CANCELLED após COMPLETED**: Regista movimento `RETURN` (peça volta ao stock ou fica danificada).

3.5.4 POST /reservations

Criar Nova Reserva

Cria uma nova reserva para uma peça. Status inicial: PENDING.

Segurança: Requer *Bearer Token* e role `SALES` ou `ADMIN`.

Regras de Validação:

- `partId`: Inteiro obrigatório. A peça deve existir e não estar eliminada.
- `notes`: String opcional para observações.

Regra de Negócio: Uma peça não pode ter mais do que uma reserva ativa (estados PENDING, IN_PREPARATION, READY_TO_SHIP ou COMPLETED).

Exemplo de Request:

```
POST /reservations
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1,
  "notes": "Cliente precisa urgente"
}
```

Resposta de Sucesso (201 Created):


```
{
  "id": 1,
  "userId": 2,
  "partId": 1,
  "assignedToId": null,
  "status": "PENDING",
  "notes": "Cliente precisa urgente",
  "cancelReason": null,
  "cancelledAt": null,
  "returnLocationId": null,
  "createdAt": "2025-12-24T10:00:00.000Z",
  "updatedAt": "2025-12-24T10:00:00.000Z",
  "user": { "id": 2, "username": "sales1", "fullName": "João Sales" },
  "part": { "id": 1, "name": "Alternador", "refInternal": "ALT-001" }
}
```


Gestão de Erros:

- **400 Bad Request**: partId em falta ou inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada (não é SALES nem ADMIN).
- **404 Not Found**: Peça não encontrada.
- **409 Conflict**: Peça já tem uma reserva ativa.
- **500 Internal Server Error**: Erro inesperado.

3.5.5 GET /reservations

Listar Reservas

Retorna a lista de reservas com filtros opcionais.

Segurança: Requer *Bearer Token* e role `SALES`, `WAREHOUSE` ou `ADMIN`.

Query Parameters (opcionais):

- `status`: Filtrar por estado (PENDING, IN_PREPARATION, READY_TO_SHIP, COMPLETED, CONFIRMED, CANCELLED).
- `userId`: Filtrar por criador da reserva.
- `assignedToId`: Filtrar por quem assumiu a preparação.

Exemplo:

```
GET /reservations?status=PENDING
GET /reservations?assignedToId=3
GET /reservations?status=COMPLETED&userId=2
```

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.5.6 GET /reservations/pending

Listar Reservas Pendentes

Retorna a lista de reservas com status PENDING, aguardando processamento pelo armazém.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.5.7 GET /reservations/my-assigned

Listar Minhas Reservas Assumidas

Retorna as reservas que o utilizador do armazém assumiu para preparação.

Segurança: Requer *Bearer Token* e role `WAREHOUSE`.

Nota: Retorna apenas reservas com status IN_PREPARATION ou READY_TO_SHIP.

Gestão de Erros:

- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada (não é WAREHOUSE).
- **500 Internal Server Error**: Erro inesperado.

3.5.8 GET /reservations/:id

Obter Detalhes de uma Reserva

Retorna os detalhes completos de uma reserva específica, incluindo informação de cancelamento se aplicável.

Segurança: Requer *Bearer Token* e role `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros de URL:

- `id`: ID da reserva (inteiro positivo).

Gestão de Erros:

- **400 Bad Request**: ID inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **404 Not Found**: Reserva não encontrada.
- **500 Internal Server Error**: Erro inesperado.

3.5.9 POST /reservations/:id/assign

Assumir Preparação de Reserva

Permite a um utilizador do armazém assumir a preparação de uma reserva pendente.

Segurança: Requer *Bearer Token* e role `WAREHOUSE`.

Parâmetros de URL:

- `id`: ID da reserva (inteiro positivo).

Efeitos:

- Campo `assignedToId` é preenchido com o ID do utilizador.
- Status muda de PENDING para IN_PREPARATION.

Gestão de Erros:

- **400 Bad Request**: ID inválido ou reserva não está PENDING.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada (não é WAREHOUSE).
- **404 Not Found**: Reserva não encontrada.
- **409 Conflict**: Reserva já foi assumida por outro utilizador.
- **500 Internal Server Error**: Erro inesperado.

3.5.10 PATCH /reservations/:id/status

Atualizar Status da Reserva

Atualiza o status de uma reserva seguindo as transições permitidas. Quando o status muda para COMPLETED, é automaticamente registado um movimento EXIT no stock.

Segurança: Requer *Bearer Token* e role `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros de URL:

- `id`: ID da reserva (inteiro positivo).

Exemplo de Request:

```
PATCH /reservations/1/status
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "READY_TO_SHIP"
}
```

Exemplo - Marcar como COMPLETED (regista EXIT automático):

```
{
  "status": "COMPLETED"
}
```

Exemplo - Confirmar receção pelo cliente:

```
{
  "status": "CONFIRMED"
}
```

Regra de Negócio: Apenas o utilizador que assumiu a reserva (`assignedTo`) pode atualizar o status, exceto para cancelamento.

Gestão de Erros:

- **400 Bad Request**: ID inválido, status inválido ou transição não permitida.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Não é o utilizador que assumiu a reserva.
- **404 Not Found**: Reserva não encontrada.
- **500 Internal Server Error**: Erro inesperado.

3.5.11 POST /reservations/:id/cancel

Cancelar Reserva

Cancela uma reserva ativa com motivo obrigatório. Para cancelamentos após COMPLETED, é automaticamente registado um movimento RETURN.

Segurança: Requer *Bearer Token* e role `SALES`, `WAREHOUSE` ou `ADMIN`.

Parâmetros de URL:

- `id`: ID da reserva (inteiro positivo).

Corpo do Request:

- `cancelReason`: Obrigatório. Valores: `DESIST`, `RETURN`, `DAMAGED_RETURN`.
- `returnLocationId`: Obrigatório apenas para `RETURN`. ID da localização de destino.

Exemplo - Desistência (antes de COMPLETED):

```
POST /reservations/1/cancel
Authorization: Bearer <token>
Content-Type: application/json

{
  "cancelReason": "DESIST"
}
```

Exemplo - Devolução normal (após COMPLETED):

```
{
  "cancelReason": "RETURN",
  "returnLocationId": 5
}
```

Exemplo - Devolução danificada (após COMPLETED):

```
{
  "cancelReason": "DAMAGED_RETURN"
}
```

Efeitos por Tipo:

- **DESIST**: Reserva cancelada, peça permanece no armazém.
- **RETURN**: Movimento RETURN registado, peça volta para `returnLocationId`.
- **DAMAGED_RETURN**: Movimento RETURN registado, peça fica `condition=DAMAGED`, `isVisible=false`, `locationId=null`.

Resposta de Sucesso (200 OK):

```
{
  "id": 1,
  "status": "CANCELLED",
  "cancelReason": "RETURN",
  "cancelledAt": "2025-12-24T15:30:00.000Z",
  "returnLocationId": 5,
  "returnLocation": {
    "id": 5,
    "fullCode": "ARM1-A-02-P1",
    "rack": "A",
    "shelf": "02",
    "pallet": "P1"
  },
  ...
}
```

Gestão de Erros:

- **400 Bad Request**:
  - ID inválido.
  - cancelReason em falta ou inválido.
  - Usar DESIST após COMPLETED.
  - Usar RETURN/DAMAGED_RETURN antes de COMPLETED.
  - RETURN sem returnLocationId.
  - Localização inválida ou sem capacidade.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **404 Not Found**: Reserva não encontrada.
- **500 Internal Server Error**: Erro inesperado.

3.6 Módulo: Movimentos de Stock (Stock Movements)

Este módulo gere todos os movimentos de inventário no armazém, incluindo entradas, saídas, transferências, devoluções e ajustes. Integra-se automaticamente com os módulos de Peças e Reservas para manter a rastreabilidade completa do stock.

3.6.1 Tipos de Movimento

MovementType

1. **ENTRY**: Entrada de peça no sistema (criação ou reposição).
2. **EXIT**: Saída de peça (reserva completada).
3. **TRANSFER**: Transferência entre localizações.
4. **ADJUSTMENT**: Ajuste manual de stock (correções administrativas).
5. **RETURN**: Devolução de peça ao stock.

3.6.2 Integração Automática

Movimentos Automáticos

- **Criar Peça (POST /parts)**: Regista automaticamente `ENTRY`.
- **Reserva COMPLETED**: Regista automaticamente `EXIT`.
- **Cancelamento após COMPLETED**: Regista automaticamente `RETURN`.

3.6.3 Modelo de Dados

```
StockMovement {
  id: Int
  partId: Int           // Peça movimentada
  userId: Int           // Utilizador que executou
  type: MovementType    // ENTRY, EXIT, TRANSFER, ADJUSTMENT, RETURN
  sourceLocId: Int?     // Localização de origem (null para ENTRY)
  destLocId: Int?       // Localização de destino (null para EXIT)
  timestamp: DateTime   // Data/hora do movimento
}
```

3.6.4 POST /stock-movements/entry

Registar Entrada de Peça

Regista a entrada de uma peça numa localização do armazém. Atualiza automaticamente o `locationId` da peça.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Nota: Este endpoint é usado manualmente. Na criação de peças via POST /parts, o ENTRY é registado automaticamente.

Corpo do Request:

- `partId`: Inteiro obrigatório. ID da peça.
- `locationId`: Inteiro obrigatório. ID da localização de destino.

Exemplo de Request:

```
POST /stock-movements/entry
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1,
  "locationId": 5
}
```

Validações:

- Localização deve existir e pertencer a um armazém ativo.
- Localização deve ter capacidade disponível.

Gestão de Erros:

- **400 Bad Request**: Campos em falta, localização inválida ou sem capacidade.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.6.5 POST /stock-movements/exit

Registar Saída de Peça

Regista a saída de uma peça do armazém. Define o `locationId` da peça como `null`.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Nota: Este endpoint é usado manualmente. Quando uma reserva muda para COMPLETED, o EXIT é registado automaticamente.

Corpo do Request:

- `partId`: Inteiro obrigatório. ID da peça.

Exemplo de Request:

```
POST /stock-movements/exit
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1
}
```

Gestão de Erros:

- **400 Bad Request**: partId em falta ou inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **404 Not Found**: Peça não encontrada.
- **500 Internal Server Error**: Erro inesperado.

3.6.6 POST /stock-movements/transfer

Transferir Peça Entre Localizações

Move uma peça de uma localização para outra dentro do armazém.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Corpo do Request:

- `partId`: Inteiro obrigatório. ID da peça.
- `fromLocationId`: Inteiro obrigatório. ID da localização de origem.
- `toLocationId`: Inteiro obrigatório. ID da localização de destino.

Exemplo de Request:

```
POST /stock-movements/transfer
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1,
  "fromLocationId": 5,
  "toLocationId": 12
}
```

Validações:

- Peça deve estar na localização de origem especificada.
- Localização de destino deve existir e ter capacidade disponível.

Gestão de Erros:

- **400 Bad Request**: Campos em falta, peça não está na origem, ou destino sem capacidade.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.6.7 POST /stock-movements/return

Registar Devolução de Peça

Regista a devolução de uma peça ao stock. Pode ser devolução normal ou danificada.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Nota: Este endpoint é usado manualmente. Ao cancelar uma reserva COMPLETED, o RETURN é registado automaticamente.

Corpo do Request:

- `partId`: Inteiro obrigatório. ID da peça.
- `toLocationId`: Inteiro obrigatório se `isDamaged=false`. ID da localização de destino.
- `isDamaged`: Boolean opcional (default: false). Se true, peça fica danificada.

Exemplo - Devolução Normal:

```
POST /stock-movements/return
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1,
  "toLocationId": 5,
  "isDamaged": false
}
```

Exemplo - Devolução Danificada:

```
{
  "partId": 1,
  "isDamaged": true
}
```

Efeitos por Tipo:

- **isDamaged=false**: Peça volta para `toLocationId`.
- **isDamaged=true**: Peça fica `condition=DAMAGED`, `isVisible=false`, `locationId=null`.

Gestão de Erros:

- **400 Bad Request**: toLocationId em falta para devolução normal, ou localização sem capacidade.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.6.8 POST /stock-movements/adjustment

Ajuste Manual de Stock

Permite correções administrativas na localização de uma peça. Apenas para ADMIN.

Segurança: Requer *Bearer Token* e role `ADMIN`.

Corpo do Request:

- `partId`: Inteiro obrigatório. ID da peça.
- `newLocationId`: Inteiro ou null. Nova localização (null para remover de localização).

Exemplo de Request:

```
POST /stock-movements/adjustment
Authorization: Bearer <token>
Content-Type: application/json

{
  "partId": 1,
  "newLocationId": 8
}
```

Exemplo - Remover de localização:

```
{
  "partId": 1,
  "newLocationId": null
}
```

Gestão de Erros:

- **400 Bad Request**: Campos em falta ou localização inválida.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada (não é ADMIN).
- **404 Not Found**: Peça não encontrada.
- **500 Internal Server Error**: Erro inesperado.

3.6.9 GET /stock-movements/part/:partId

Histórico de Movimentos de uma Peça

Retorna todos os movimentos de stock de uma peça específica, ordenados por data decrescente.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Parâmetros de URL:

- `partId`: ID da peça (inteiro positivo).

Exemplo:

```
GET /stock-movements/part/1
Authorization: Bearer <token>
```

Resposta de Sucesso (200 OK):

```
[
  {
    "id": 5,
    "partId": 1,
    "userId": 3,
    "type": "EXIT",
    "sourceLocId": 5,
    "destLocId": null,
    "timestamp": "2025-12-24T15:00:00.000Z",
    "user": { "id": 3, "username": "warehouse1", "fullName": "Maria Armazém" },
    "sourceLoc": {
```

```
      "id": 5,
      "fullCode": "ARM1-A-01-P1",
      "warehouse": { "id": 1, "code": "ARM1", "name": "Armazém Principal" }
    },
    "destLoc": null
  },
  {
    "id": 1,
    "type": "ENTRY",
    "sourceLocId": null,
    "destLocId": 5,
    ...
  }
]
```

Gestão de Erros:

- **400 Bad Request**: partId inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.6.10 GET /stock-movements/recent

Movimentos Recentes (Dashboard)

Retorna os movimentos mais recentes de todo o sistema, útil para dashboards e monitorização.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Query Parameters (opcionais):

- `limit`: Número máximo de resultados (1-200, default: 50).

Exemplo:

```
GET /stock-movements/recent?limit=20
Authorization: Bearer <token>
```

Resposta de Sucesso (200 OK):

```
[
  {
    "id": 10,
    "type": "TRANSFER",
```

```
    "timestamp": "2025-12-24T16:30:00.000Z",
    "part": { "id": 3, "name": "Motor de Arranque", "refInternal": "MA-003" },
    "user": { "id": 3, "username": "warehouse1", "fullName": "Maria Armazém" },
    "sourceLoc": { "id": 5, "fullCode": "ARM1-A-01-P1", ... },
    "destLoc": { "id": 12, "fullCode": "ARM1-B-02-P3", ... }
  },
  ...
]
```

Gestão de Erros:

- **400 Bad Request**: limit fora do intervalo permitido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.6.11 GET /stock-movements/locations

Listar Localizações Disponíveis

Retorna a lista de localizações com informação detalhada para seleção em UI. Inclui dados do armazém, rack, shelf, pallet e capacidade disponível.

Segurança: Requer *Bearer Token* e role `WAREHOUSE` ou `ADMIN`.

Query Parameters (opcionais):

- `warehouseId`: Filtrar por armazém específico.

Exemplo:

```
GET /stock-movements/locations
GET /stock-movements/locations?warehouseId=1
Authorization: Bearer <token>
```

Resposta de Sucesso (200 OK):

```
[
  {
    "id": 5,
    "fullCode": "ARM1-A-01-P1",
    "rack": "A",
    "shelf": "01",
    "pallet": "P1",
    "warehouse": { "id": 1, "code": "ARM1", "name": "Armazém Principal" },
    "capacity": 10,
    "currentParts": 7,
```

```
    "availableSpace": 3,
    "hasSpace": true
  },
  {
    "id": 6,
    "fullCode": "ARM1-A-01-P2",
    "rack": "A",
    "shelf": "01",
    "pallet": "P2",
    "warehouse": { "id": 1, "code": "ARM1", "name": "Armazém Principal" },
    "capacity": 5,
    "currentParts": 5,
    "availableSpace": 0,
    "hasSpace": false
  },
  ...
]
```


Nota para Frontend: Este endpoint fornece todos os dados necessários para construir uma UI de seleção de localização user-friendly, com dropdowns hierárquicos (Armazém → Rack → Shelf → Pallet) ou lista com códigos legíveis.

Gestão de Erros:

- **400 Bad Request**: warehouseId inválido.
- **401 Unauthorized**: Token inválido ou não fornecido.
- **403 Forbidden**: Role não autorizada.
- **500 Internal Server Error**: Erro inesperado.

3.7 Módulo: Auditoria (Audit Logs)

Permite aos administradores consultar todos os registos produzidos pelo módulo 5. Os endpoints são read-only e protegidos por role `ADMIN`.

3.7.1 GET /audit-logs

Listar Logs com Filtros

Devolve uma lista paginada de eventos, incluindo metadados do utilizador que originou cada ação.

Segurança: Requer *Bearer Token* e role `ADMIN`. Requests sem autenticação retornam 401; roles diferentes de ADMIN recebem 403.

Parâmetros de Query (opcionais):

- `page` (default: 1) e `pageSize` (default: 25, máx. 200).
- `userId`: inteiro positivo para filtrar logs de um utilizador específico.
- `action`, `entity`, `entityId`: strings livres com *contains* case-insensitive.
- `search`: pesquisa simultânea em `action`, `entity` e `entityId`.
- `dateFrom`, `dateTo`: datas ISO 8601 (UTC) para definir o intervalo temporal.

Resposta de Sucesso (200 OK):

```
{
 "items": [
  {
   "id": 120,
   "action": "PART_CREATE",
   "entity": "PART",
   "entityId": "58",
   "details": { "ref": "TEST-001" },
   "timestamp": "2026-01-02T10:11:12.000Z",
   "user": { "id": 1, "username": "admin", "role": "ADMIN" }
  }
 ],
 "total": 42,
 "page": 1,
 "pageSize": 25,
 "totalPages": 2
}
```

Gestão de Erros:

- **400 Bad Request**: validação Zod falhou (ex.: data inválida, pageSize >200).
- **401 Unauthorized**: token ausente/expirado.
- **403 Forbidden**: role diferente de ADMIN.
- **500 Internal Server Error**: erro inesperado na consulta.

3.7.2 GET /audit-logs/:id

Consultar Log Específico

Retorna todos os detalhes de um registo único, útil para investigações ou exportação manual.

Segurança: Igual ao endpoint anterior (Bearer + ADMIN).

Resposta de Sucesso (200 OK):


```
{
 "id": 120,
 "action": "USER_PASSWORD_RESET",
 "entity": "USER",
 "entityId": "5",
 "details": {
  "reason": "ADMIN_RESET",
  "performedBy": 1
 },
 "timestamp": "2026-01-02T12:00:00.000Z",
 "user": {
  "id": 1,
  "username": "admin",
  "fullName": "Administrador",
  "role": "ADMIN"
 }
}
```


Gestão de Erros:

- **400 Bad Request**: `id` inválido (não numérico ou <1).
- **401 / 403**: conforme descrito acima.
- **404 Not Found**: registo inexistente.
- **500 Internal Server Error**: erro inesperado ao consultar a base de dados.